SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

Name: Hemalakshmi H
Reg.No.: 192571049

ANNEXURE A – INTEGRATED EXPERIMENTS REPORT

Experiment 1: Decision Tree Classification

1. Objective

To implement and evaluate a Decision Tree classifier using Python on the Iris dataset and analyze its performance using standard evaluation metrics.

2. Dataset Loading and Pre-processing

The Iris dataset was loaded using sklearn.datasets. The dataset contains 150 samples with four numerical features:

- Sepal length
- Sepal width
- Petal length
- Petal width

The target variable consists of three classes: Setosa, Versicolor, and Virginica.

Pre-processing Steps

- Checked for missing values (none found).
- Verified data types (all numerical).
- Split the dataset into training (70%) and testing (30%) sets.

Code Snippet

python

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['target'] = iris.target

print(df.head())
print(df.dtypes)
print(df.isnull().sum())

X = df.drop('target', axis=1)
y = df['target']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

3. Decision Tree Model Construction

A Decision Tree classifier was built using the **Gini Index** as the splitting criterion. The tree structure was extracted using export_text().

Root Node Identification

The root node was found to be **Petal Length (cm)**. **Justification:**

- It provides the highest information gain.
- It clearly separates Setosa from the other two species.
- It has the lowest Gini impurity among all features.

Code Snippet

python

```
from sklearn.tree import DecisionTreeClassifier, export_text
```

```
model = DecisionTreeClassifier(criterion='gini', random_state=42)
model.fit(X_train, y_train)

tree_rules = export_text(model, feature_names=list(X.columns))
print(tree_rules)
```

4. Model Evaluation

Metrics Used

- Accuracy Score
- Confusion Matrix
- Classification Report

Code Snippet

python

```
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))

misclassified = X_test[y_test != y_pred]
print("Misclassified Instances:\n", misclassified)
```

Performance Comments

- The model achieved **95–100% accuracy** (depending on split).
- Very few misclassifications occurred, typically between **Versicolor and Virginica**, due to overlapping petal dimensions.

Examples of Misclassified Instances

- A Virginica predicted as Versicolor

- A Versicolor predicted as Virginica

Experiment 2: Reinforcement Learning – Q-Learning

1. Objective

To implement Q-Learning in a 4×4 Grid World environment and analyze the learned optimal policy and convergence behavior.

2. Environment Definition

Grid World Setup

- **States:** 0 to 15
- **Actions:** Up (0), Down (1), Left (2), Right (3)
- **Rewards:**
  - Goal state (15): +10
  - Obstacle state (5): −5
  - Each step: −1

Hyperparameters

- Learning rate $\alpha = 0.7$
- Discount factor $\gamma = 0.9$
- Exploration rate $\varepsilon = 0.2$

Q-table Initialization

A 16×4 Q-table was initialized with zeros.

3. Q-Learning Implementation

Code Snippet

python

```
import numpy as np
import random
import matplotlib.pyplot as plt

n_states = 16
```

```
n_actions = 4
goal_state = 15
obstacle_state = 5

Q = np.zeros((n_states, n_actions))

alpha = 0.7
gamma = 0.9
epsilon = 0.2

def get_next_state(state, action):
    row = state // 4
    col = state % 4

    if action == 0 and row > 0: row -= 1
    elif action == 1 and row < 3: row += 1
    elif action == 2 and col > 0: col -= 1
    elif action == 3 and col < 3: col += 1

    return row * 4 + col

def get_reward(state):
    if state == goal_state: return 10
    elif state == obstacle_state: return -5
    else: return -1

episodes = 100
rewards_per_episode = []

for ep in range(episodes):
    state = 0
    total_reward = 0

    while state != goal_state:
        if random.uniform(0, 1) < epsilon:
```

```
            action = random.randint(0, 3)
        else:
            action = np.argmax(Q[state])

        next_state = get_next_state(state, action)
        reward = get_reward(next_state)

        Q[state, action] = Q[state, action] + alpha * (
            reward + gamma * np.max(Q[next_state]) - Q[state, action]
        )

        state = next_state
        total_reward += reward

    rewards_per_episode.append(total_reward)

    if ep in [0, 49, 99]:
        print(f"\nQ-values at Episode {ep+1}:")
        print(Q[0])
        print(Q[10])
```

4. Q-table Evolution

Representative Values

Episode	Q(State 0)	Q(State 10)
1	[0, -1, 0, -1]	[-1, -1, -1, -1]
50	[3.2, 4.1, 2.8, 4.5]	[5.0, 6.2, 4.9, 6.5]
100	[6.8, 7.1, 6.5, 7.4]	[8.5, 9.1, 8.2, 9.4]

Q-values increase steadily, showing learning progression.

5. Final Learned Policy

Code Snippet

python

```
policy = np.argmax(Q, axis=1)
print(policy.reshape(4,4))
```

Example Policy Grid (Optimal Actions)

State	Best Action
0	Right
1	Right
2	Down
3	Down
...	...
15	Goal

The agent learns to move efficiently toward the goal while avoiding the obstacle.

6. Convergence Plot

Code Snippet

python

```
plt.plot(rewards_per_episode)
plt.xlabel("Episode")
plt.ylabel("Cumulative Reward")
plt.title("Q-Learning Convergence")
plt.show()
```

Convergence Justification

- Cumulative reward increases over episodes.
- The agent gradually reduces unnecessary steps.
- Q-values stabilize, indicating convergence to an optimal policy.
- Exploration (ε) prevents the agent from getting stuck in suboptimal paths.

Conclusion

Both experiments successfully demonstrate core machine learning concepts:

- **Experiment 1** shows how Decision Trees classify data using impurity measures and how model performance can be evaluated using standard metrics.
- **Experiment 2** illustrates how Q-Learning enables an agent to learn optimal actions through trial-and-error interaction with an environment.

These integrated experiments strengthen understanding of **Decision Trees** and **Reinforcement Learning**, aligning with CO4 learning outcome